Introduction & Elementary Structures

CSE 4303: Data Structures
Lecture 01

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

November 9, 2025

Lecture Overview

1. Introduction
2. Classification
3. Review of Arrays, Records, Pointers
4. Memory Allocations
5. End

🤔 What is a Data Structure?

Data

Simply values or sets of values, raw facts or figure without any specific meaning.

🤔 What is a Data Structure?

Data

Simply values or sets of values, raw facts or figure without any specific meaning.

Data Structure

The logical or mathematical model of a particular organization of data. A specific way of organizing, storing, and managing data in a computer's memory.

🤔 What is a Data Structure?

Data

Simply values or sets of values, raw facts or figure without any specific meaning.

Data Structure

The logical or mathematical model of a particular organization of data. A specific way of organizing, storing, and managing data in a computer's memory.

It's about two things:

1. How to store data (structure)
2. How to perform operations on it (efficiency)

💡 The "What" vs. The "How"

The "What": Abstract Data Type (ADT)

An ADT is a mathematical model that defines a set of data and the operations on that data, without specifying how they are implemented.

- 👉 What operations are performed, not how they are performed.
- 👉 Operation on Stack –
  - `push()`
  - `pop()`

The "How":Data Structure (DS)

A DS is a concrete implementation of data storage that allows us to efficiently perform the operations defined by an ADT.

- 👉 How data is organized and stored in memory.
- 👉 Stack implementation –
  - Array →fixed size, random access
  - Linked List →dynamic size,

🤔 Why Study Data Structure?

Applications of Data Structure

- 👉 Computer file system (Data structure maps file names onto hard drive sectors)
- 👉 Google and other search engines (Data structure maps keywords on web pages containing those keywords)
- 👉 What is the longest common subsequence of two DNA can be found?
- 👉 Geographic systems (Data structure find data relevant to the current view/location)
- 👉 Block chain (Linked list)
- 👉 Google Map (Finding shortest distances in terms of distance and time)
- 👉 Data Compression (Huffman's encoding)

Many problems are solved efficiently just using the right data structure. This course is about learning to choose the right data structure for the job.

🤔 How Do We Study Data Structures?

We focus on

👉 The logical structure — how data elements relate to each other.
Computer file system (data structure maps file names onto hard drive track and sectors)

🤔 How Do We Study Data Structures?

We focus on

👉 The logical structure — how data elements relate to each other.
Computer file system (data structure maps file names onto hard drive track and sectors)

👉 The operations — which operations it supports:

- Reading: accessing at a particular element within the data structure.
- Searching: looking for a particular value within a data structure.
- Inserting: adding a new value to the data structure.
- Deleting: removing a value from the data structure.
- Sorting: rearranging element in some logical order.

🤔 How Do We Study Data Structures?

We focus on

👉 The logical structure — how data elements relate to each other.
Computer file system (data structure maps file names onto hard drive track and sectors)

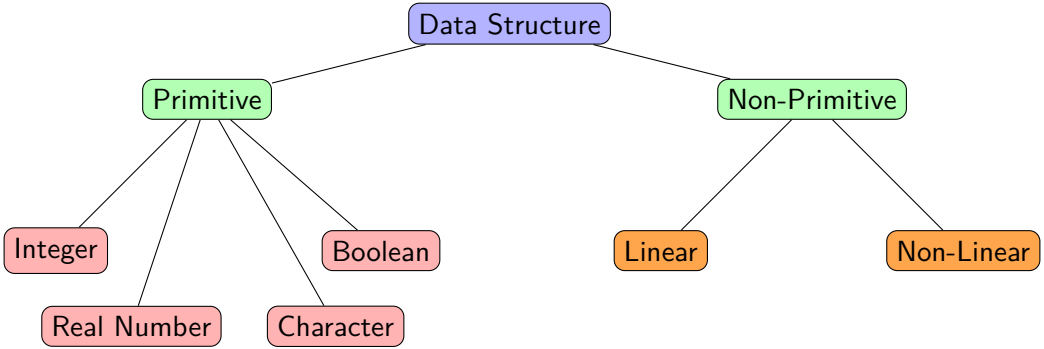
👉 The operations — which operations it supports:

- Reading: accessing at a particular element within the data structure.
- Searching: looking for a particular value within a data structure.
- Inserting: adding a new value to the data structure.
- Deleting: removing a value from the data structure.
- Sorting: rearranging element in some logical order.

👉 The efficiency — how efficiently allowed operations can be done.
Searching an element in:

- An **unsorted array** takes $O(n)$ time.
- A **balanced binary search tree** takes $O(\log n)$ time.

Classification of Data Structure



Non-Premive Example:

Linear: Arrays, Linked List, Stack, Queue

Non-Linear: Graph, Trees

🧠 Review of Arrays, Records, Pointers I

Arrays:

A collection of elements of the same type stored in contiguous memory locations.

Characteristics:

- 👉 Fixed size defined at declaration
- 👉 Elements accessed using index (starting from 0)
- 👉 All elements are of the same type

Advantages:

- 👉 Fast access via index
- 👉 Simple memory allocation

Disadvantages:

- 👉 Fixed size, cannot grow dynamically
- 👉 Insertion/deletion in middle is costly

🧠 Review of Arrays, Records, Pointers II

Records (Structures):

A collection of related data items of different types grouped under a single name.

Characteristics:

- 👉 Can store heterogeneous data
- 👉 Useful for modeling real-world entities

Advantages:

- 👉 Groups related data together
- 👉 Improves program readability and organization

Disadvantages:

- 👉 Access requires dot operator
- 👉 Not suitable for very large collections without arrays of records

🧠 Review of Arrays, Records, Pointers III

Pointers:

A variable that stores the memory address of another variable.

Characteristics:

- 👉 Enables dynamic memory allocation
- 👉 Used in linked data structures like linked lists, trees, and graphs

Advantages:

- 👉 Efficient memory management
- 👉 Supports dynamic data structures
- 👉 Can pass large structures to functions without copying

Disadvantages:

- 👉 Risk of memory leaks if not managed properly
- 👉 Pointer arithmetic can be unsafe

Memory Allocations I

Memory allocation can be classified into followings:

- Contiguous memory allocation

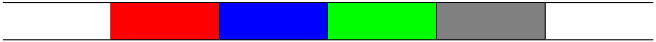


Figure: Contiguous Memory Block (array)

- Linked memory allocation

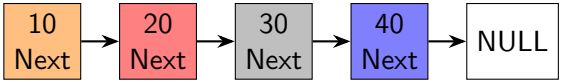


Figure: Linked Memory (Linked List)

Memory Allocations II

- Indexed memory allocation

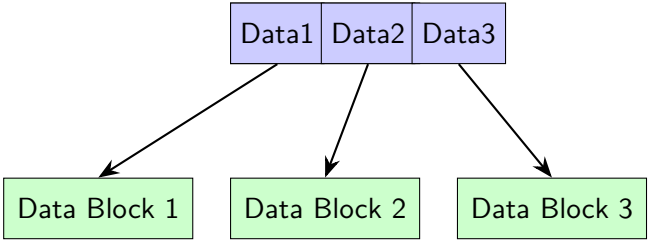


Figure: Indexed memory allocation (Array of pointers)

🚃 Contiguous Memory Allocation

An array stores n objects in a single contiguous space of memory.

- 👉 Can directly access any point randomly. Random access is possible.
- 👉 Unfortunately, if more memory is required, a request for new memory usually requires copying all information into the new memory.
- 👉 In general, you cannot request for the operating system to allocate to you the next n memory locations

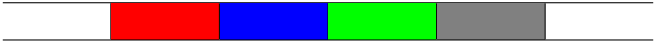


Figure: Contiguous Memory Block (array)

Linked Memory Allocation

Linked storage such as a linked list associates two pieces of data with each item being stored:

- The object itself, and
- A reference to the next item
- Random access to any data apart from the beginning is not possible since the address of a particular data is only stored to its previous data.

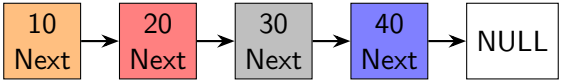


Figure: Linked Memory (Linked List)

Indexed Memory Allocation

An array of pointers (possibly NULL) link to a sequence of allocated memory locations.

Pros:

- No external fragmentation: Data blocks don't need to be contiguous
- Direct access possible: Accessing the index block, you can directly access any block without traversing other blocks.
- Scalability: Large files can grow easily; add more data blocks and update the index block.

Cons:

- Extra memory for index block: Each file needs a separate index block.
- Slight overhead: Accessing a data block requires first reading the index block.

Example: File storing in disk/drives.

🤝 Conclusion & Q&A

Thank You!

👋 Thank you for your time and attention. 👋

🗨 Any questions or clarifications. 💭